



PROJEKT: DATA-MART- ERSTELLUNG IN SQL

Abschlussbericht

B. Sc. Informatik
30.06.2026
Korin Lohmeyer
IU14091946
Dr. Anna Androvitsanea

Inhalt

Funktionalität der Datenbank.....	1
Erweiterte Integritätsregeln.....	2
Performancebetrachtung	2
Reflexion	3
Datenschutz und Sicherheit.....	4
Metadaten	4
Installationsanweisung	5

Abschlussbericht

Funktionalität der Datenbank

Die entwickelte Datenbank bildet die Grundlage einer Buchtausch-Anwendung und ermöglicht die Verwaltung hierfür relevanter Informationen. Hierzu gehören Informationen über die Bücher als auch Informationen zu den Nutzern, sowie logische Ergänzungen um Funktionen wie Versand, Abholung, Ausleihe und Bewertung abbilden zu können. Durch die entwickelte Struktur können einzelne Informationen miteinander verknüpft werden, wodurch komplexe Zusammenhänge entstehen.

Die Grundlegende Struktur bildet hierbei ein Buch mit entsprechenden Informationen, wie Autor, Verlag, Sprache, Genre und Titel ab. Von einem Buch kann es mehrere Exemplare geben, welche von unterschiedlichen Nutzern besessen werden. Das Buchindividuum besitzt dabei wiederum einen Zustand. Also die Verfassung in den es sich befindet. Dieses Individuum ist ein Nutzer zugeordnet. Ein Nutzer besitzt Informationen wie Adresse und Kontaktinformationen. Durch diese Informationen werden dann unterschiedliche Funktionen abgeleitet.

Ein Buchindividuum wird ein Verfügbarkeitszeitraum zugeordnet, welcher bestimmt, wann das Buch ausgeliehen werden kann. Nur wenn ein Exemplar hier aufgeführt ist, kann es dann tatsächlich ausgeliehen werden. Bei einer Ausleihe werden dann Startzeit wie Endzeit als auch der Ausleihende sowie das explizite Exemplar aufgeführt. Durch dieses Exemplar ist der Besitzer des Buches ebenso erkenntlich. Durch die Ausleihe besteht dann die Möglichkeit noch eine Bewertung zu der Ausleihe und dem Exemplar zu verfassen. Diese Bewertung ist nur möglich, wenn eine tatsächliche Ausleihe stattgefunden hat. Zu den Prozess der Ausleihe gehört zusätzlich, in einer getrennten Relation, die Art und Weise wie das Exemplar zu den Ausleihenden gelangt. Also durch Versand oder durch Abholung. In diesen Relationen werden noch zusätzliche Informationen gehalten, wie eine Sendungsnummer oder der Abholzeitpunkt.

Neben den eigentlichen Relationen wurden noch drei unterschiedliche Views implementiert, um komplexe Zusammenhänge über mehrere Relationen vereinfacht darzustellen. Hierzu gehört eine Übersicht über Bewertungen, eine Übersicht über Ausleihen und eine Übersicht über Verfügbare Bücher.

Erweiterte Integritätsregeln

Im Rahmen der Finalisierung wurden zusätzliche Integritätsregeln direkt in der Datenbank umgesetzt. Ziel dieser Regeln ist es, fehlerhafte oder fachlich unzulässige Datensätze bereits beim Einfügen oder Ändern von Daten zu verhindern. Dadurch wird die Datenqualität unabhängig von der späteren Anwendung sichergestellt.

Hierzu wurden verschiedene CHECK-Constraints eingeführt. So wird beispielsweise geprüft, dass Bewertungen ausschließlich Werte zwischen 1 und 10 besitzen können. Zusätzlich wird sowohl bei der Ausleihe als auch bei der Buchverfügbarkeit sichergestellt, dass das Enddatum immer nach dem Startdatum liegt. Des Weiteren dürfen Hausnummern ausschließlich positive Werte besitzen und Versandkosten keine negativen Werte annehmen.

Neben den CHECK-Constraints tragen auch Primär- und Fremdschlüssel zur Datenintegrität bei. Primärschlüssel sorgen dafür, dass jeder Datensatz eindeutig identifiziert werden kann. Fremdschlüssel stellen sicher, dass Beziehungen nur zwischen bereits vorhandenen Datensätzen bestehen. Dadurch kann beispielsweise keine Ausleihe für ein nicht existierendes Buchexemplar oder einen nicht vorhandenen Nutzer angelegt werden.

Nicht alle fachlichen Regeln lassen sich jedoch sinnvoll ausschließlich durch Constraints umsetzen. Komplexere Anforderungen, wie beispielsweise die Vermeidung überlappender Verfügbarkeitszeiträume eines Buchexemplars, würden den Einsatz von Triggern oder Exclusion Constraints erfordern. Da diese Mechanismen den Umfang der Arbeit deutlich erweitert hätten, wurde auf deren Implementierung verzichtet.

Performancebetrachtung

Neben der eigentlichen Funktionalität wurde auch die Performance der Datenbank betrachtet. Zwar besteht die Datenbank im Rahmen dieser Arbeit lediglich aus Testdaten, dennoch wurde bereits berücksichtigt, wie sich eine größere Datenmenge auf spätere Abfragen auswirken könnte.

Hierzu wurden zusätzliche Indexe auf häufig verwendeten Fremdschlüsseln angelegt. Diese werden insbesondere bei JOIN-Operationen und Suchabfragen genutzt. Ohne einen Index müsste die Datenbank bei jeder Abfrage die komplette Tabelle durchsuchen. Durch die angelegten Indexe kann PostgreSQL die benötigten Datensätze deutlich schneller finden. Primärschlüssel sowie eindeutige Attribute werden von PostgreSQL bereits automatisch indexiert und mussten daher nicht zusätzlich berücksichtigt werden.

Auch die entwickelten Views profitieren von den angelegten Indexen. Da diese mehrere Tabellen miteinander verknüpfen, können die entsprechenden JOIN-Abfragen effizienter ausgeführt werden. Gleichzeitig erhöhen die Views die Übersichtlichkeit, da häufig verwendete Abfragen nicht jedes Mal neu formuliert werden müssen.

Ein direkter Performancevergleich wurde im Rahmen dieser Arbeit nicht durchgeführt, da die verwendeten Testdaten hierfür zu klein sind. Die eingeführten Indexe bilden jedoch bereits eine Grundlage dafür, dass die Datenbank auch bei einer deutlich größeren Anzahl an Datensätzen effizient arbeiten kann.

Reflexion

Bei der Erstellung des Projektes sind zusätzlich einige Schwierigkeiten aufgetreten. Bei der Implementierung musste sehr genau darauf geachtet werden, in welcher Reihenfolge die Tabellen eingefügt werden. Dies ist wichtig, da das Nutzen eines Fremdschlüssels ohne bestehende Referenz zu einem Fehler führt. Des Weiteren haben kleinere Syntaxfehler den Implementierungsprozess aufgehalten. Das Debugging mit DBeaver war anfangs ungewohnt hat aber schließlich den Prozess beschleunigt.

Anfangs war nicht klar, welche Fremdschlüssel in welche Tabelle gehören. Hierdurch werden in diesem Datenbankmodell viele Beziehungen moduliert. Insbesondere bei den Beziehungen zwischen Ausleihe, Buch Verfügbar, Versand und Abholung musste mehrfach überarbeitet werden. Dadurch wird klar, wie wichtig eine gründliche Planung vor der Implementierung ist. Fehler in der Planung würden die Datenbank fachlich Inkorrekt machen.

Die Testdatenerfassung war ebenfalls ein Problem. Zunächst habe ich versucht mir passende Daten im Internet zu suchen. Dieser Versuch hat vieles verkompliziert, wodurch beschlossen wurde, die anfänglichen Daten zu verwerfen. Die Buchdaten stammen aus Büchern, die tatsächlich in meinem Besitz sind. Die Nutzerdaten hingegen wollte ich mit einem persönlichen Touch gestalten. Sie bestehen aus Persönlichkeiten, welche ich bewundere oder auch eine Buch Protagonistin. Eine weitere Schwierigkeit war, dass die Daten durchgehend konsistent sein müssen. Wenn in der Entität Buchverfügbar das Attribut *Post* auf wahr gesetzt wurde, muss dieses Buch auch bei einer Ausleihe in der Tabelle Versand aufgenommen werden, wenn es ausgeliehen wird. Ähnliches Muster besteht, bei der Entität Bewertung. Diese muss auch an eine tatsächliche Ausleihe gebunden sein.

Mögliche Erweiterungen können sein, dass die Funktion geschaffen wird, dass Bücher reserviert werden können. Eine weitere Erweiterung ist eine automatische Detektion einer

Überlappung der Verfügbarkeitszeiträumen. Zusätzlich wäre eine Einführung von Rollen in dem System denkbar.

Datenschutz und Sicherheit

Da in der entwickelten Datenbank personenbezogene Daten wie Namen, E-Mail-Adressen und Anschriften gespeichert werden, müssen die Vorgaben der DSGVO berücksichtigt werden. Ziel hierbei ist es, diese Daten vor unbefugtem Zugriff zu schützen.

Hierzu gibt es verschiedene Möglichkeiten. Beispielsweise kann die Verbindung zwischen Anwendung und Datenbank verschlüsselt werden, sodass die Daten nicht einfach mitgelesen werden können. Außerdem sollte der Zugriff auf die Datenbank nur autorisierten Personen möglich sein. PostgreSQL bietet hierfür ein Rollen- und Rechtekonzept. Dadurch kann festgelegt werden, welche Benutzer Daten lesen, bearbeiten oder löschen dürfen.

Im Rahmen dieser Arbeit wurde ein solches Rollen- und Rechtekonzept nicht umgesetzt, da der Schwerpunkt auf der Modellierung und Implementierung der Datenbank lag. Für einen produktiven Einsatz wäre dies jedoch sinnvoll. Zusätzlich sollten regelmäßige Backups erstellt werden, um Datenverluste zu vermeiden. Auch der Server, auf dem die Datenbank betrieben wird, sollte regelmäßig gewartet werden und die Anforderungen der DSGVO erfüllen.

Metadaten

Tabelle 1: Metadaten der Datenbank

Daten	Wert
Anzahl Entitäten	15
Anzahl Views	3
Anzahl Indexe	18
Größe der Datenbank	8747 kB
Anzahl der Testdaten	158
Anzahl der Check-Constraints	5

Installationsanweisung

Für diese Datenbank wurde als Datenbankmanagementsystem PostgreSQL verwendet. Um die Relationen zu erstellen, muss die SQL-Datei *01_create_tables.sql* ausgeführt werden. Hierdurch werden alle Tabellen erstellt. Anschließend werden Daten hinzugefügt in dem die Datei *02_insert_data.sql* ausgeführt wird. Die einzelnen Views werden durch Ausführen der einzelnen View-Dateien angelegt. Die Indexe werden durch das Ausführen der Datei *03_create_index.sql* angelegt. Die Datenbank steht anschließend vollständig mit Testdaten zur Verfügung.

Zur Dokumentation wurde ebenfalls ein Github-Repository angelegt:

<https://github.com/Xp3x/BuchtauschDatenbank>